

Моделирование распространения ударной волны в газе с локальным энерговложением средствами библиотеки JAX

Докладчик: Филькин Д. С.

Научный руководитель: Кравченко О. В.

группа ФН1-81Б

6 июня 2026 г.



Мотивация

Для выявления физических закономерностей динамики ударной волны требуется проведение параметрического исследования с параметром разрежения α

Цель работы

Численное моделирование распространения ударной волны в газе с локальным энерговыделением средствами разработанного высокопроизводительного программного комплекса на базе библиотеки JAX.

Основные задачи:

- Высокопроизводительная реализация центральных схем высокого разрешения SD2 и FD2 средствами JAX,
- кросс-верификация решения относительно библиотек centpy и JAX-FLUIDS,
- анализ аппаратного ускорения на CPU и GPU,
- демонстрация возможностей комплекса на задаче о прохождении УВ через зону энерговыделения в 1D и 2D.

Библиотека JAX — инструмент реализации

Что такое JAX

JAX — современная библиотека Google для высокопроизводительных вычислений и машинного обучения; NumPy-совместимый, JIT-компиляция.

Почему именно JAX

Параметрическое исследование по α требует множества быстрых расчетов. JAX дает скорость компилируемого кода при простоте Python, а также единый код CPU/GPU.

Альтернативы и их ограничения

Инструмент	CPU/GPU	Единый код	Порог входа
NumPy / SciPy	CPU	—	низкий
Fortran, C++/CUDA	CPU+GPU	нет	высокий
Numba, CuPy	CPU / GPU	частично	средний
PyTorch, TensorFlow	CPU+GPU	да	средний
JAX	CPU+GPU	да	низкий

JAX объединяет производительность GPU с

простотой NumPy.

Центральные схемы второго порядка

SD2 — полудискретная

Реконструкция:

$$u_{i\pm 1/2}^{\mp} = u_i \pm \frac{1}{2} \min\text{mod}(u_i - u_{i-1}, u_{i+1} - u_i)$$

Численный поток:

$$H_{i+1/2} = \frac{1}{2} (f(u_{i+1/2}^+) + f(u_{i+1/2}^-) - a_{i+1/2}(u_{i+1/2}^+ - u_{i+1/2}^-))$$

Полудискретная ОДУ:

$$\frac{du_i}{dt} = - \frac{H_{i+1/2} - H_{i-1/2}}{\Delta x}$$

интегрирование по времени — SSP-RK2

FD2 — полностью дискретная

Наклоны:

$$u'_i = \min\text{mod}(u_i - u_{i-1}, u_{i+1} - u_i) \\ f'_i = \min\text{mod}(f_i - f_{i-1}, f_{i+1} - f_i)$$

Предиктор:

$$u_i^{n+1/2} = u_i^n - \frac{\Delta t}{2\Delta x} f'_i$$

Корректор (шахматная сетка):

$$u_{i+1/2}^{n+1} = \frac{1}{2} (u_i^n + u_{i+1}^n) + \frac{1}{8} (u'_i - u'_{i+1}) - \frac{\Delta t}{\Delta x} (f(u_{i+1}^{n+1/2}) - f(u_i^{n+1/2}))$$

Одномерная постановка задачи

Система уравнений Эйлера

$$\partial_t \begin{pmatrix} \rho \\ \rho u \\ \rho E \end{pmatrix} + \partial_x \begin{pmatrix} \rho u \\ \rho u^2 + p \\ (\rho E + p)u \end{pmatrix} = 0$$

с уравнением состояния идеального газа

$$p = (\gamma - 1) \left(\rho E - \frac{1}{2} \rho u^2 \right), \quad \gamma = 1.4$$

в расчетной области $x \in [0, 1]$, с кусочно-постоянными НУ

$$(\rho, u, p)|_{t=0} = \begin{cases} (1, 0, 1), & \text{при } x \in [0, 0.3), \\ (\alpha \cdot 1, 0, 1), & \text{при } x \in [0.3, 0.5], \\ (1, 0, 1), & \text{при } x \in (0.5, 0.8), \\ (2.667, -1.479, 4.5), & \text{при } x \in [0.8, 1.0]. \end{cases}$$

Зона пониженной плотности при $\alpha = 0.1$ моделирует энерговложение;
Слева — ГУ Неймана, справа — ГУ непротекания.

Численное решение: одномерная постановка

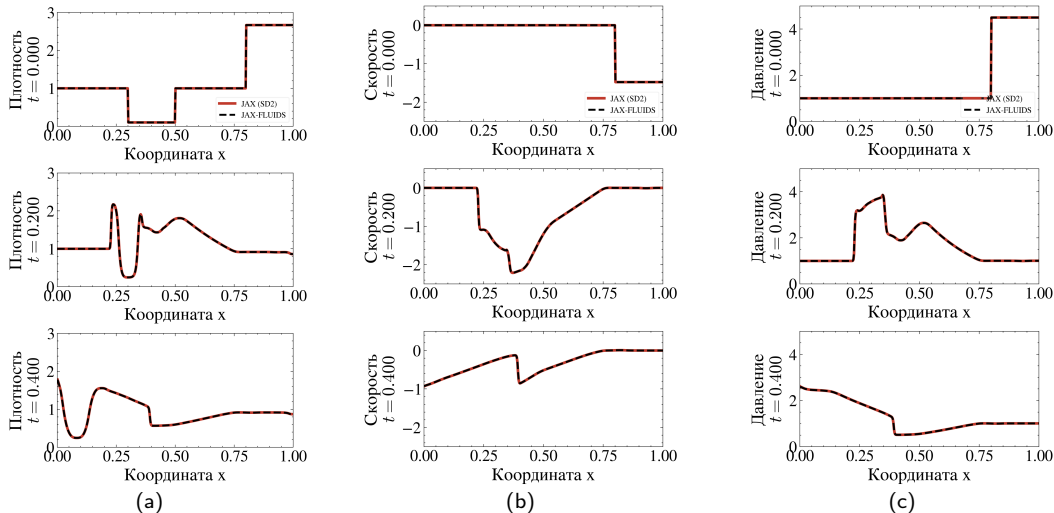


Рис. Эволюция примитивных переменных в модифицированной 1D задаче Римана: (a) Плотность; (b) Скорость; (c) Давление с наложением на эталон JAX-FLUIDS

Анализ сходимости в одномерной постановке

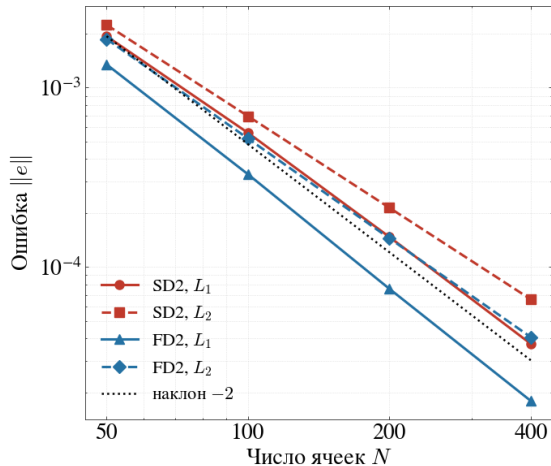


Рис. Ошибка по плотности ρ . Гладкая задача, $t = 1$, ограничитель тс.

Тест: перенос гладкого профиля плотности

$$\rho(x, 0) = 1 + 0.2 \sin(2\pi x), \quad u = 1, \quad p = 1$$

$$\rho(x, t) = 1 + 0.2 \sin(2\pi(x - t))$$

Таблица Эмпирический порядок p ($N: 200 \rightarrow 400$)

Схема	p_{L_1}	p_{L_2}
SD2	1.98	1.70
FD2	2.08	1.83

Анализ аппаратного ускорения: одномерная постановка

Таблица Время счета (с), 1D, схемы SD2 и FD2

J	centpy		JAX CPU		JAX GPU		JAX-FL.	
	SD2	FD2	SD2	FD2	SD2	FD2	CPU	GPU
256	1.138	0.218	0.096	0.023	0.143	0.131	0.954	3.894
512	4.244	0.662	0.394	0.057	0.179	0.189	2.857	7.545
1024	5.577	1.972	2.499	0.207	0.370	0.280	4.893	16.403
2048	16.175	3.875	7.255	0.785	0.720	0.588	13.040	33.451
4096	41.613	13.218	29.928	4.067	1.500	1.217	44.688	70.443
8192	152.440	46.947	86.862	14.377	3.931	3.082	161.556	153.630
16384	612.826	186.817	349.892	60.462	12.805	8.819	633.204	404.548

Таблица Ускорение JAX GPU (1D, $J = 16384$)

Сравнение	SD2	FD2
JAX CPU	27.3×	6.9×
centpy	47.9×	21.2×
JAX-FLUIDS GPU	31.6×	45.9×

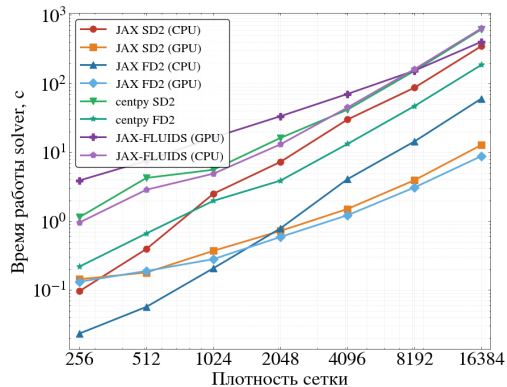


Рис. Время счета (с) от размера сетки J , 1D

Система уравнений Эйлера (2D)

$$\partial_t \begin{pmatrix} \rho \\ \rho u \\ \rho v \\ \rho E \end{pmatrix} + \partial_x \begin{pmatrix} \rho u \\ \rho u^2 + p \\ \rho uv \\ (\rho E + p)u \end{pmatrix} + \partial_y \begin{pmatrix} \rho v \\ \rho uv \\ \rho v^2 + p \\ (\rho E + p)v \end{pmatrix} = 0$$

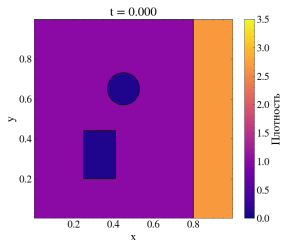
с тем же уравнением состояния идеального газа $p = (\gamma - 1)(\rho E - \frac{1}{2}\rho(u^2 + v^2))$, $\gamma = 1.4$.

в расчетной области $(x, y) \in [0, 1]^2$, с кусочно-постоянными НУ

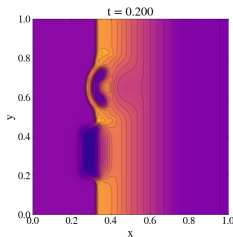
$$(\rho, u, v, p)|_{t=0} = \begin{cases} (1, 0, 0, 1), & \text{невозмущенный газ,} \\ (\alpha \cdot 1, 0, 0, 1), & \text{зоны энерговложения,} \\ (2.667, -1.479, 0, 4.5), & \text{ударная волна: } x \in [0.8, 1]. \end{cases}$$

Зоны энерговложения при $\alpha = 0.1$ — круговая (центр $(0.45, 0.65)$, радиус 0.08) и прямоугольная ($x \in [0.25, 0.41]$, $y \in [0.20, 0.44]$); слева — ГУ Неймана, остальные — ГУ непротекания.

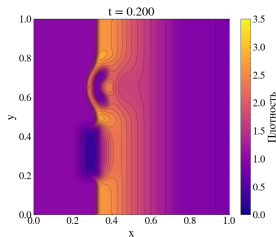
Двумерная постановка: эволюция плотности



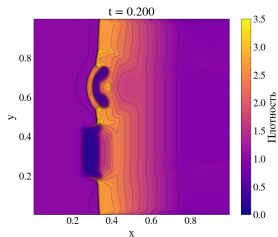
Начальное условие



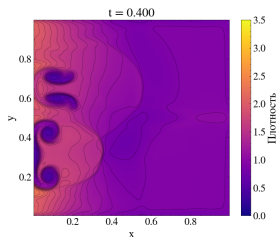
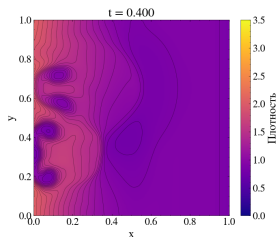
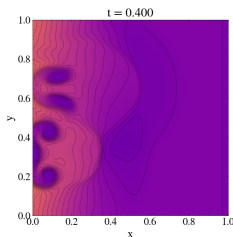
SD2



FD2



JAX-FLUIDS



Анализ сходимости в двумерной постановке

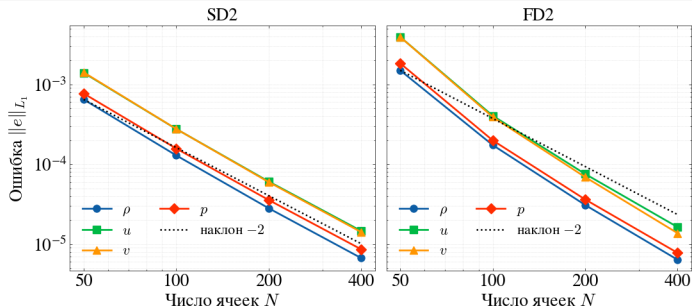


Рис. Ошибка по ρ, p, u, v в норме L_1 , 2D

Таблица Эмпирический порядок p ($N: 200 \rightarrow 400$)

Перем.	SD2		FD2	
	p_{L_1}	p_{L_2}	p_{L_1}	p_{L_2}
ρ	2.07	2.06	2.28	2.29
u	2.06	2.08	2.21	2.26
v	2.08	2.09	2.33	2.38
p	2.05	2.07	2.22	2.30

Тест: перенос гладкого изоэнтропийного вихря Ху–Шу ($\varepsilon = 5$) на области $[0, 10]^2$ с периодическими ГУ; неограниченная реконструкция, $t = 1$.

Анализ аппаратного ускорения: двумерная постановка

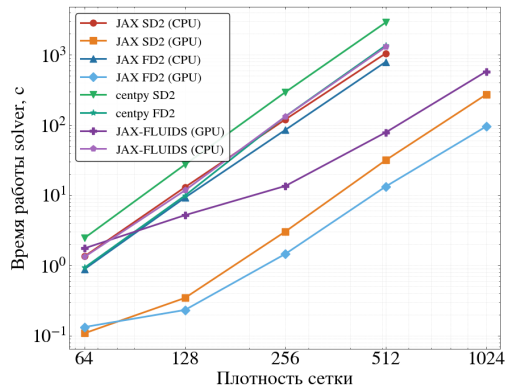


Рис. Время счета (с) от размера сетки $J \times J$, 2D

Таблица Время счета (с), 2D, схемы SD2 и FD2

$J \times J$	centpy		JAX CPU		JAX GPU		JAX-FL.	
	SD2	FD2	SD2	FD2	SD2	FD2	CPU	GPU
64	2.456	0.930	1.348	0.885	0.108	0.133	1.322	1.753
128	27.274	9.865	12.988	9.233	0.344	0.232	11.709	5.191
256	293.400	132.367	119.955	85.072	3.036	1.458	131.803	13.530
512	2888.498	1357.738	1047.884	793.620	31.572	13.316	1294.979	78.417
1024	—	—	—	—	271.457	95.325	—	577.726

Таблица Ускорение JAX GPU (2D, 512×512)

Сравнение	SD2	FD2
JAX CPU	33.2×	59.6×
centpy	91.5×	102.0×
JAX-FLUIDS GPU	2.5×	5.9×

Выводы

- 1 Поставлена и решена модифицированная задача Римана о прохождении УВ через зону энерговложения для уравнений Эйлера (1D и 2D).
- 2 Реализован высокопроизводительный комплекс на базе JAX (схемы SD2 и FD2): JIT-компиляция, векторизация, единый код для CPU и GPU без модификации.
- 3 Подтвержден второй порядок точности на гладких решениях.
- 4 Проведена кросс-верификация с точным решением, centry и JAX-FLUIDS: достигнуто хорошее согласие профилей решений.
- 5 В двумерных задачах решатель на GPU быстрее собственной CPU-реализации до $33\times$ (SD2) и $60\times$ (FD2), быстрее centry в $90\text{--}100\times$, быстрее JAX-FLUIDS в $2.5\text{--}6\times$; достигнута сетка 1024×1024 .

Нормы погрешности L_1 , L_2 и порядок сходимости

Для количественной оценки точности используются нормы L_1 и L_2 .

Норма L_1 — погрешность интегрального характера

$$\|e\|_{L_1} = \frac{1}{N} \sum_{i=1}^N |u_i^h - u_i^{\text{ref}}|$$

Норма L_2 — среднеквадратическая (равномерность погрешности)

$$\|e\|_{L_2} = \sqrt{\frac{1}{N} \sum_{i=1}^N (u_i^h - u_i^{\text{ref}})^2}$$

где u_i^h — численное решение в центре i -й ячейки сетки из N ячеек, u_i^{ref} — точное аналитическое решение.

Эмпирический порядок сходимости (формула Ричардсона)

Как работает JAX (JIT-конвейер)

- ❶ **Трассировка:** JAX проходит по Python-функции с абстрактными аргументами и записывает все операции.
- ❷ **Граф (jaxpr):** из трассы строится промежуточное представление — граф вычислений.
- ❸ **Компиляция XLA:** граф транслируется в оптимизированное машинное ядро под устройство (CPU/GPU/TPU).
- ❹ **Исполнение и кэш:** ядро запускается; повторные вызовы берут готовую компиляцию из кэша.

Для преодоления вычислительных ограничений языка Python в качестве основы решателя выбран высокопроизводительный фреймворк JAX.

Ключевые архитектурные преимущества:

- **JIT-компиляция (Just-In-Time):** компилятор XLA (Accelerated Linear Algebra) транслирует высокоуровневый код Python в оптимизированные машинные инструкции.
- **Глубокая векторизация:** отказ от явных циклов по пространственной сетке; массивы обрабатываются параллельно на уровне тензорных операций.
- **Кроссплатформенность:** единый код (математическая логика решателя) нативно и без модификаций выполняется на многоядерных CPU и графических ускорителях (GPU/TPU).

Основные особенности метода:

- **Вычисление потоков:** потоки на гранях ячеек определяются без вычисления матриц Якоби и характеристических переменных.

Подходы к дискретизации:

- **Fully-Discrete (FD):** пространственный и временной шаги жестко связаны единым вычислительным шаблоном.
- **Semi-Discrete (SD):** пространственная дискретизация сводит УЧП к системе ОДУ (метод прямых). Интегрирование по времени выполняется независимыми TVD-схемами Рунге–Кутты. Различают схемы 2-го (SD2) и 3-го (SD3) порядка точности.

TVD-свойство (Total Variation Diminishing)

Схема не порождает новые локальные экстремумы, если

$$TV(u^{n+1}) \leq TV(u^n),$$

где общая вариация решения $TV(u) = \sum_{i=1}^{N-1} |u_{i+1}(t) - u_i(t)|$

Для обеспечения TVD-свойства используются нелинейные ограничители наклонов (limiter):

$$\text{minmod}(a, b) = \frac{1}{2} (\text{sgn}(a) + \text{sgn}(b)) \cdot \min(|a|, |b|).$$

Производительность: одномерная задача (бэкап)

Таблица Время счета (с), 1D, схема SD2

J	centpy	Разраб. CPU	Разраб. GPU	JAX-FL. CPU	JAX-FL. GPU
256	1.138	0.096	0.143	0.954	3.894
512	4.244	0.394	0.179	2.857	7.545
1024	5.577	2.499	0.370	4.893	16.403
2048	16.175	7.255	0.720	13.040	33.451
4096	41.613	29.928	1.500	44.688	70.443
8192	152.440	86.862	3.931	161.556	153.630
16384	612.826	349.892	12.805	633.204	404.548

Ускорение GPU (1D, $J = 16384$): $47.9\times$ относительно centpy, $27.3\times$ относительно собственной CPU-версии. На малых 1D-сетках выигрыш GPU невелик из-за накладных расходов; преимущество раскрывается в 2D.